La filosofía **Lean** en desarrollo de productos de software se centra en la optimización integral del sistema productivo mediante la eliminación sistemática del desperdicio (*muda*) y la maximización del valor entregado al usuario final.

Principio	Explicación
1. Eliminar desperdicios	Identificar y eliminar actividades que no aportan valor.
2. Amplificar el aprendizaje	Aprender continuamente mediante feedback y experimentación.
3. Embeber la integridad conceptual	Mantener la coherencia y calidad del producto en su conjunto.
4. Diferir compromisos	Evitar tomar decisiones irreversibles cuando todavía existe demasiada incertidumbre.
5. Dar poder al equipo	Permitir que las personas que realizan el trabajo participen en las decisiones.
6. Ver el todo	Optimizar el sistema completo y no únicamente actividades individuales.
7. Entregar lo antes posible	Obtener valor y retroalimentación rápidamente.

Lean-Kanban se fundamenta en los siguientes ejes:

- **Foco en el cliente**
- **Flujo continuo:** Establecimiento de un tránsito uniforme y sin interrupciones de las unidades de trabajo a lo largo de la cadena de valor, y minimizando tareas improductivas.
- **Eliminación de desperdicios (*muda*):** En el desarrollo de software, el desperdicio no se limita únicamente a los **defectos**, sino que abarca fundamentalmente la **acumulación de inventario en colas** (trabajo en proceso o *WIP* no finalizado) y los **tiempos muertos o de espera** entre transiciones de fase dentro del flujo de valor.
- **Liderazgo, Entendimiento, Acuerdo y Respeto:** Promoción del compromiso organizacional orientado a la mejora continua mediante el análisis sistemático de los procesos vigentes, el consenso de la colectividad sobre los cambios a implementar y la valoración respetuosa de las responsabilidades e individuos existentes.

1.2 Método Kanban, Políticas Explícitas, WIP Limits y Gestión de Colas

El método Kanban es un marco operacional evolutivo guiado por el flujo de trabajo que busca optimizar la entrega de servicios de software mediante la visibilidad operacional y la autorregulación.

Principios de Kanban

Principios de Gestión de Cambios:

- **Comenzar con lo que haces ahora:** respetar los procesos actuales tal como se ejecutan en la práctica.
- **Acordar la búsqueda de la mejora a través del cambio evolutivo:** Guiar el avance del sistema mediante modificaciones graduales democráticamente.
- **Fomentar actos de liderazgo a todos los niveles**

Principios de Entrega de Servicios:

- **Comprender y enfocarse en cumplir las necesidades y expectativas del cliente**

Políticas Explícitas

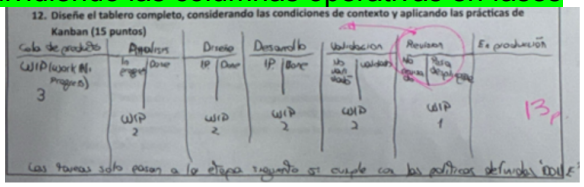
Para garantizar la previsibilidad, transparencia y gobierno del flujo, el sistema Kanban exige:

- **Reposición de trabajo en el tablero:** Reglas que establecen las condiciones, la frecuencia, las cantidades permitidas y las autoridades responsables para alimentar la cola de trabajo.
- **Criterios de tracción (*pull criteria* / *Definition of Done*):** Criterios rigurosos de entrada y salida que condicionan la tracción de una tarea desde un estado del flujo al subsiguiente una vez completados los estándares de calidad definidos.
- **Límites de trabajo en curso (*WIP Limits*):** Restricciones cuantitativas máximas fijadas para acotar la cantidad de trabajo simultáneo permitido en cada columna o estado del sistema.
- **Clases de servicio:** Políticas diferenciadas diseñadas para tratar las unidades de trabajo de acuerdo a su nivel de urgencia, costo de demora o impacto del riesgo.

Límites de Trabajo en Curso (WIP Limits) y Gestión de Colas

La implantación de *WIP limits* previene la sobrecarga del sistema productivo, obliga al equipo a finalizar tareas antes de absorber nuevo trabajo (*stop starting, start finishing*) y saca a la luz los cuellos de botella subyacentes en el flujo.

El tablero Kanban estructura el flujo de trabajo dividiendo las columnas operativas en fases con subsistemas de *En Progreso* y *Hecho* :



1.3 Métricas en Enfoques Tradicional, Lean y Agile

Enfoque	Métrica / Concepto	Definición / Fórmula Formal
Lean / Kanban	**Lead Time**	Tiempo total transcurrido desde la formulación formal del requerimiento por parte del cliente hasta su entrega operativa final.
	Cycle Time	Intervalo en el que el equipo de desarrollo ejecuta activamente tareas para resolver el requerimiento desde que inicia su procesamiento.
	Touch Time	Tiempo efectivo durante el cual se realiza trabajo directo de valor agregado sobre el artefacto de software.
	Elapsed Time	Tiempo total del proceso que incluye tanto el trabajo activo (*Touch Time*) como los tiempos muertos y esperas en cola.
	% Eficiencia del Ciclo de Proceso	$\%\ \text{Eficiencia del Ciclo de Proceso} = \frac{\text{Touch Time}}{\text{Elapsed Time}}$
	Métricas orientadas a servicio	• **Expectativa de nivel de servicio:** Desempeño esperado por el cliente. • **Capacidad del nivel de servicio:** Rendimiento real ejecutable por el sistema. • **Acuerdo de nivel de servicio (SLA):** Compromiso formal pactado. • **Umbral de adecuación:** Nivel mínimo aceptable por debajo del cual el servicio es deficiente.
Agile	**Velocidad**	Volumen de unidades de trabajo o puntos de historia completados en estado de 'Listo' (*Done*) durante una iteración.
	Capacidad	Disponibilidad real de esfuerzo del equipo para comprometer trabajo en el horizonte de la siguiente iteración.
	Release Burn-down / RTF	Gráficos de trabajo remanente contra el tiempo para proyectar entregas (*Burn-down*) y tasa de funcionalidades listas para producción (*RTF*).
Tradicional	**Métricas Fijas de Avance**	Medición estática basada en el porcentaje teórico de avance físico por etapas lineales secuenciales.
	Esfuerzo Planificado vs. Real	Contraste cuantitativo de horas-hombre y recursos financieros presupuestados contra la ejecución real contabilizada.

2. Unidad 3: Gestión de Software como Producto y Prácticas Continuas

Habiendo establecido las bases del control del flujo de trabajo, la ingeniería de software moderna concibe la construcción de sistemas no como proyectos cerrados, sino como la evolución continua de un **producto**. Esta perspectiva exige la adopción de un modelo empírico de control de procesos y la articulación de prácticas continuas que garanticen una retroalimentación ágil, minimicen los riesgos del proyecto y reduzcan drásticamente los costos de corrección en el software.

2.1 Mejora Empírica y Feedback Continuo

El desarrollo de software como producto opera en entornos caracterizados por una alta complejidad y una dinámica constante en los requerimientos del mercado. Frente a la rigidez de la planificación predictiva tradicional, la gestión moderna se fundamenta en el **control de procesos empíricos**, el cual descansa sobre tres pilares conceptuales: **transparencia, inspección y adaptación** .

La implementación de ciclos cortos de retroalimentación (*feedback continuo*) resulta indispensable para gestionar la incertidumbre de manera sistemática. Estos ciclos permiten someter hipótesis de producto a validación constante con usuarios y partes interesadas, inspeccionar incrementos funcionales del sistema y adaptar la estrategia operacional de forma inmediata.

Desde la perspectiva de la calidad del producto, el ciclo empírico de mejora continua —alineado con la dinámica **Plan-Do-Check-Act (PDCA)** — incide directamente sobre la **Curva del Costo de Cambio** (*Cost of Change Curve*). Al acortar el tiempo entre la concepción de una funcionalidad, su implementación y su validación, las inconsistencias conceptuales y los defectos arquitectónicos se detectan en etapas tempranas. Esto evita la propagación del error hacia fases avanzadas del desarrollo, reduciendo en varios órdenes de magnitud el esfuerzo y el costo económico requerido para corregir el producto de software.

2.2 Integración de Testing Continuo

Dentro de la ingeniería continua de productos de software, las actividades de pruebas sufren un cambio estructural: dejan de constituir un filtro tardío, masivo y aislado —históricamente posicionado al término de la fase de codificación— para transformarse en una **actividad de ingeniería transversal e integrada de manera permanente** en todo el ciclo de vida de desarrollo.
El testing continuo implica la automatización exhaustiva de pruebas y la verificación ininterrumpida de criterios de calidad en cada commit o incremento funcional. Esta práctica asegura que cada pequeña modificación en la base de código sea evaluada inmediatamente en términos de correctitud funcional, desempeño y seguridad. De este modo, la calidad del producto trasciende la visión de responsabilidad de un área aislada de aseguramiento de calidad (QA) para convertirse en una responsabilidad técnica compartida por todo el equipo de desarrollo, garantizando la disponibilidad de un producto utilizable y potencialmente desplegable en cualquier momento.

3. Unidad 4: Aseguramiento de Calidad de Proceso y de Producto

3.1 Conceptos Generales de Calidad y PPQA (Process and Product Quality Assurance)

Definiciones Institucionales de Calidad de Software

- **ISTQB (International Software Testing Qualifications Board):** Proceso involucrado en la planificación, preparación y evaluación de productos de software y artefactos de trabajo relacionados, con el objetivo de determinar que satisfacen los requerimientos especificados, demostrar que son aptos para el propósito (*fit for purpose*) y detectar defectos.
- **IEEE Std 610-1990:** El proceso de operar un sistema o componente bajo condiciones especificadas, observar o registrar los resultados y realizar una evaluación de algún aspecto del sistema o componente.
- **IEEE Std 829-1998:** El proceso de analizar un elemento de software para detectar las diferencias entre las condiciones existentes y las requeridas (es decir, fallas/errores) y evaluar las características de los elementos de software.

Aseguramiento de la Calidad de Proceso y de Producto (PPQA)

El área de proceso **PPQA** tiene como objetivo primordial proporcionar a la organización y a la dirección ejecutiva una visibilidad objetiva e imparcial sobre los procesos utilizados y los productos de trabajo generados a lo largo del proyecto. PPQA opera de manera independiente para auditar analíticamente dos dimensiones complementarias:

1. **Calidad de Proceso:** Evaluación objetiva de que las actividades de gestión e ingeniería del software se ejecutan en estricto cumplimiento de los estándares, metodologías, descripciones de proceso y procedimientos institucionales establecidos.
2. **Calidad de Producto de Trabajo:** Verificación formal e independiente de que los artefactos intermedios y finales (documentos de especificación, arquitectura, planes, código y suites de prueba) satisfacen los criterios de calidad y los requisitos definidos antes de su liberación formal.

Ventajas:

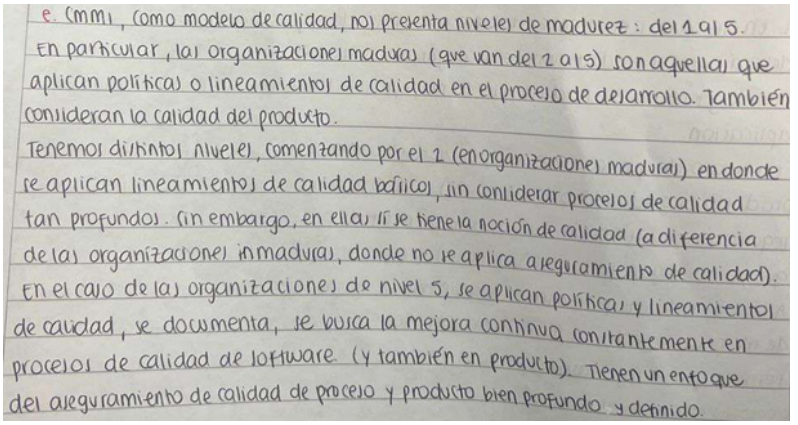
- Suministra a la alta gerencia e inversores una visibilidad objetiva e independiente sobre el apego real a los procesos del proyecto.
- Asegura la identificación temprana de desviaciones y no conformidades frente a la normativa y estándares vigentes.
- Institucionaliza las buenas prácticas de ingeniería de software, promoviendo la mejora continua de los procesos organizacionales.
- Garantiza que ningún artefacto de trabajo o versión del producto sea liberado sin haber superado los controles de calidad requeridos.

Desventajas:

- Riesgo de introducir rigidez burocrática y sobrecarga de gestión documental si las auditorías no se calibran según la complejidad del proyecto.
- Incurre en costos operacionales asociados a la necesidad de contar con personal capacitado e independiente dedicado al aseguramiento de calidad.
- Puede originar fricción cultural o resistencia en los equipos si la función de PPQA es percibida como una auditoría punitiva y no como una actividad colaborativa de mejora.

3.2 Modelos de Calidad de Software: CMMI, SPICE e ISO/IEC 25000

Modelo / Estándar	Enfoque Principal	Niveles / Estructura	Aplicación en la Ingeniería
CMMI *(Capability Maturity Model Integration)*	Evaluación y mejora de la madurez de los procesos organizacionales y la capacidad de las áreas de proceso.	**Representación Escalonada:** 5 Niveles de Madurez (1. Inicial, 2. Gestionado, 3. Definido, 4. Gestionado Cuantitativamente, 5. En Optimización). **Representación Continua:** Niveles de Capacidad por área.	Diagnóstico institucional, estandarización de procesos de ingeniería y evaluación formal de la capacidad operativa de las organizaciones.
SPICE *(ISO/IEC 15504)*	Marco de referencia para la evaluación y determinación de la capacidad de los procesos de desarrollo de software.	**Niveles de Capacidad de Proceso:** Nivel 0 (Incompleto), Nivel 1 (Realizado), Nivel 2 (Gestionado), Nivel 3 (Establecido), Nivel 4 (Predecible), Nivel 5 (En Optimización).	Evaluación objetiva de procesos de software para la selección rigurosa de proveedores de TI y la conducción de planes de mejora interna.
ISO/IEC 25000 *(SQuaRE - Software Product Quality Requirements and Evaluation)*	Evaluación cuantitativa y especificación de la calidad del **producto de software.**	**Estructurado en 5 divisiones:** Gestión de la Calidad (2500n), Modelo de Calidad (2501n), Medición de Calidad (2502n), Requisitos de Calidad (2503n) y Evaluación de Calidad (2504n).	Definición, medición y certificación cuantitativa de las características internas, externas y de calidad en uso del software (funcionalidad, usabilidad, eficiencia, mantenibilidad, etc.).


e. CMMI, como modelo de calidad, no presenta niveles de madurez: del 1 al 5.
En particular, las organizaciones maduras (que van del 2 al 5) son aquellas que aplican políticas o lineamientos de calidad en el proceso de desarrollo. También consideran la calidad del producto.
Tenemos distintos niveles, comenzando por el 2 (en organizaciones madura) en donde se aplican lineamientos de calidad básicos, sin considerar procesos de calidad tan profundos. Sin embargo, en ellas ya se tiene la noción de calidad (a diferencia de las organizaciones inmaduras, donde no se aplica aseguramiento de calidad).
En el caso de las organizaciones de nivel 5, se aplican políticas y lineamientos de calidad, se documenta, se busca la mejora continua constantemente en procesos de calidad de software (y también en producto). Tienen un enfoque del aseguramiento de calidad de proceso y producto bien profundo y definido.

3.3 Auditorías de Software

Las auditorías son evaluaciones formales e independientes conducidas para verificar de forma objetiva la efectividad y el cumplimiento de las normativas, planes y entregables definidos.

1. **Auditoría de Proyecto:** Evaluación que examina de manera imparcial la conformidad del proyecto con respecto a los planes aprobados, presupuestos, cronogramas, metodologías y procesos de gestión institucional aplicables.
2. **Auditoría de Configuración Funcional (FCA - *Functional Configuration Audit*):** Verificación formal e independiente orientada a auditar que un elemento de configuración de software ha alcanzado de forma comprobable el conjunto completo de sus requerimientos funcionales y características de desempeño especificadas en la documentación técnica.
3. **Auditoría de Configuración Física (PCA - *Physical Configuration Audit*):** Verificación formal de ingeniería que comprueba que la construcción física real del software entregado (código ejecutable, binarios, bibliotecas y componentes instalables) coincida fielmente con la documentación de diseño y la estructura del producto declarada.

3.4 Calidad de Producto, Planificación y Niveles de Pruebas

Propósito del Testing y Calidad de Producto

Las actividades de prueba evalúan analíticamente la calidad del software con tres propósitos de ingeniería esenciales:

1. Demostrar la aptitud de uso del sistema (*fit for purpose*).
2. Detectar objetivamente las diferencias entre las condiciones observadas y las condiciones requeridas especificadas.

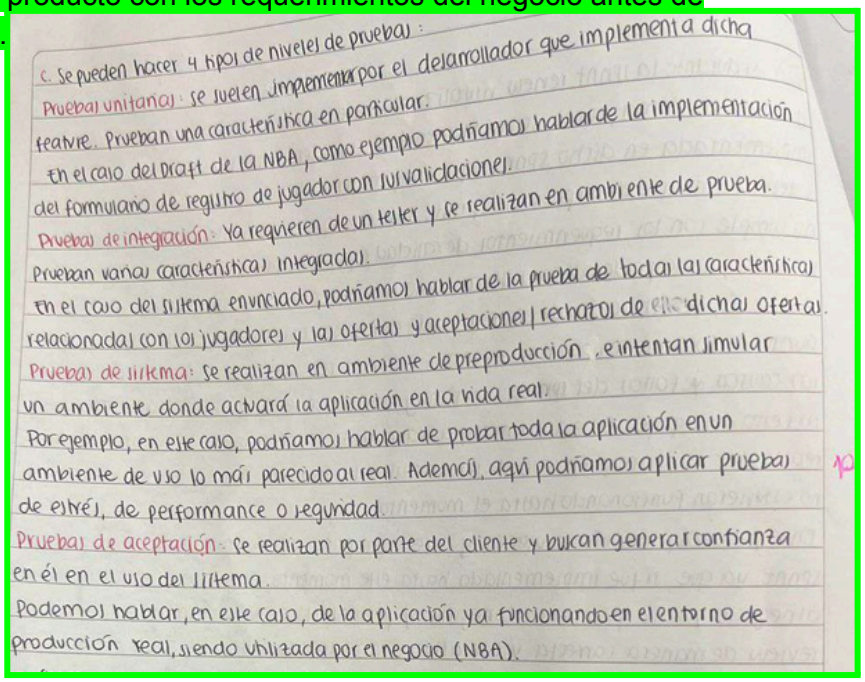
3. Identificar defectos para mitigar el riesgo de fallas operativas en entornos de producción.

Planificación de Pruebas

Fase estratégica inicial donde se establecen de forma documentada los objetivos, el alcance, las estrategias de prueba basadas en riesgos, los criterios de aceptación y finalización, la asignación de recursos, la configuración de entornos y el cronograma de ejecución del testing.

Niveles de Pruebas

- **Pruebas Unitarias:** Verificación del funcionamiento aislado de las componentes más pequeñas del código fuente (funciones, métodos o clases).
- **Pruebas de Integración:** Validación de las interfaces, protocolos de comunicación e interacción lógica entre módulos o subsistemas agrupados.
- **Pruebas de Sistema:** Constituyen la verificación integral de la aplicación cuando esta funciona como un todo estructurado (**Prueba de la Construcción Final**).
  -Determinan si el sistema opera satisfactoriamente en su globalidad.
  -Exigencia de entorno: El entorno de pruebas debe corresponder al entorno de producción tanto como sea posible, con el propósito explícito de reducir al mínimo el riesgo de incidentes originados por particularidades de la infraestructura que no hayan sido detectadas previamente.
  -Deben investigar exhaustivamente tanto **requerimientos funcionales** como **requerimientos no funcionales** del sistema (evaluando capacidad de recuperación ante fallas, seguridad y protección, comportamiento bajo estrés y métricas de desempeño o *performance*).
- **Pruebas de Aceptación:** Evaluación ejecutada por los clientes o usuarios finales para validar la conformidad del producto con los requerimientos del negocio antes de autorizar su paso a producción.


c. Se pueden hacer 4 tipos de niveles de prueba:
Pruebas unitarias: se suelen implementar por el desarrollador que implementa dicha feature. Prueban una característica en particular.
En el caso del Draft de la NBA, como ejemplo podríamos hablar de la implementación del formulario de registro de jugador con sus validaciones.
Pruebas de integración: Ya requieren de un tester y se realizan en ambiente de prueba. Prueban varias características integradas.
En el caso del sistema anunciado, podríamos hablar de la prueba de todas las características relacionadas con los jugadores y las ofertas y aceptaciones/rechazos de dichas ofertas.
Pruebas de sistema: se realizan en ambiente de preproducción e intentan simular un ambiente donde actuará la aplicación en la vida real.
Por ejemplo, en este caso, podríamos hablar de probar toda la aplicación en un ambiente de uso lo más parecido al real. Además, aquí podríamos aplicar pruebas de estrés, de performance o seguridad.
Pruebas de aceptación: Se realizan por parte del cliente y buscan generar confianza en él en el uso del sistema.
Podemos hablar, en este caso, de la aplicación ya funcionando en el entorno de producción real, siendo utilizada por el negocio (NBA).

3.5 Técnicas de Pruebas de Caja Negra (Black-Box Testing)

Concepto

El testing de Caja Negra es un enfoque funcional de diseño de pruebas que examina la conducta externa del software mediante el análisis de sus entradas y salidas, prescindiendo de todo conocimiento sobre la estructura lógica interna, la arquitectura o el código fuente del programa.

Imposibilidad del Testing Exhaustivo

La ejecución de pruebas sobre la totalidad del espacio de estados y combinaciones de entrada de un sistema es inviable.
Debido a la imposibilidad de ejecutar pruebas exhaustivas, la ingeniería de software utiliza técnicas formales de optimización para seleccionar el subconjunto con mayor probabilidad de detectar defectos:

1. **Partición de Equivalencias:** Técnica que divide el dominio de entrada de un dato en clases equivalentes válidas e inválidas, asumiendo que el sistema procesará de forma idéntica cualquier valor representativo perteneciente a la misma clase.
2. **Análisis de Valores Límites (AVL):** Técnica complementaria orientada a diseñar casos de prueba seleccionando valores específicos situados en las fronteras de las clases de equivalencia (valores mínimos, máximos y sus inmediatos adyacentes), donde estadísticamente se concentra la mayor tasa de errores de desarrollo.

3.6 Revisiones Técnicas Formales y Métricas

Tipos de Revisiones Técnicas

- **Walkthroughs (Revisiones Guiadas):** Mecanismos de revisión informal dirigidos por el propio autor del artefacto, enfocados en explicar la solución técnica al equipo y recopilar comentarios generales.
- **Inspecciones Formales:** Procesos rigurosos y altamente estructurados de evaluación técnica que persiguen la detección objetiva de defectos mediante el apego a un proceso definido, el uso de listas de chequeo y la adopción de roles formales preestablecidos.

Artefactos a Revisar y Participantes Involucrados

Artefacto a Revisar	Roles y Participantes Involucrados
Diseño detallado	Diseñador, arquitecto, programadores, testers.
Planes de proyecto	Líder de proyecto, stakeholders, representante de ventas o marketing, líder técnico, representante del área de calidad.
Especificación de requerimientos	Analista de requerimientos, líder de proyecto, arquitecto, diseñador, testers, representante de ventas y/o marketing.
Código fuente	Programador, diseñador, testers, analista de requerimientos.
Plan de testing	Tester, programador, arquitecto, diseñador, representante del área de calidad, analista de requerimientos.

Métricas Asociadas a Revisiones Técnicas Formales

- **Esfuerzo de revisión:** Horas-hombre totales consumidas durante las etapas de preparación individual y reunión de inspección.

- **Densidad de defectos:** Proporción de defectos detectados por unidad de tamaño del artefacto evaluado (ej. defectos/KLOC o defectos/página de requerimientos).
- **Tasa de preparación:** Cantidad de código o material analizado individualmente por los revisores por unidad de tiempo antes de la reunión formal.
- **Tasa de inspección:** Velocidad o volumen de artefacto evaluado en conjunto durante la sesión formal de revisión.

3.7 Testing en Ambientes Ágiles

Dimensión de Análisis	Testing Tradicional (Waterfall)	Testing en Ambientes Ágiles
Integración en el Ciclo de Vida	Proceso secuencial y lineal posicionado al final del ciclo de desarrollo (`Requirements` → `Design` → `Implementation` → `Verification`).	Proceso iterativo e integrado continuamente en cada ciclo (`Plan` → `Design` → `Implement` → `Test`).
Oportunidad de Ejecución	El testing se ejecuta como una fase tardía una vez finalizada la codificación completa del sistema.	El testing ocurre en paralelo a la codificación dentro del mismo Sprint o iteración sobre pequeños incrementos.
Ciclo de Retroalimentación	Retroalimentación lenta y diferida; los defectos se descubren semanas o meses después de ser diseñados.	Retroalimentación inmediata; las desviaciones se detectan e imprevistos se corrigen en cuestión de horas.
Costo y Riesgo de Defectos	Alto costo de corrección al detectar fallas en fases avanzadas; elevado riesgo de retraso en la fecha de liberación.	Bajo costo de corrección; el error se subsana inmediatamente dentro del marco de la misma iteración.
Responsabilidad de Calidad	Asignada de forma aislada e independientemente al equipo o departamento de QA/Testing.	Responsabilidad colectiva y compartida de todo el equipo multiprofesional (desarrolladores, testers, PO).

Principios y Valores del Tester Ágil

- Colaboración continua y directa con desarrolladores y representantes del negocio (*Product Owner*).
- Provisión de retroalimentación inmediata sobre la calidad funcional y técnica del software.
- Enfoque prioritario en la **prevención de defectos** mediante el refinado de requerimientos, en lugar de la mera detección tardía.
- Comprensión profunda del valor de negocio entregado desde la perspectiva del usuario final.

Prácticas, Rol del Tester y Desafíos Culturales

- **Prácticas de Testing Ágil:** Automatización de pruebas a múltiples niveles (unitarias, integración, aceptación), desarrollo guiado por pruebas de aceptación (ATDD/BDD), integración continua y ejecución de testing exploratorio guiado por charters.

- **Rol y Competencias del Tester:** El tester evoluciona desde un perfil ejecutor de scripts hacia un rol de **facilitador de calidad** . Requiere habilidades analíticas avanzadas, competencia técnica para colaborar en la arquitectura de automatización y habilidades blandas para refinar criterios de aceptación junto con los analistas y el negocio.
- **Desafíos Culturales:** Transición organizacional obligatoria para desmantelar la cultura de "silos funcionales". Supone desplazar la mentalidad de traspaso de responsabilidad ("el software es de QA para que busque fallas") hacia una cultura donde la responsabilidad sobre la entrega de un producto funcional de calidad sea asumida colectivamente por todo el equipo.